模倣学習によるロボットアームの Pick and Place

― 学習ステップ数の比較と、閉ループでのみ現れた3つの欠陥 ―

Physical AI講座2026 応用編1 最終課題　説明資料 Omnicampus アカウント名：ko8ya4ken

---

1. 概要

ロボットアーム **CRANE+ V2** に、机上の5cm木製ブロックを掴んで別の位置へ運ばせた。シミュレータは Gazebo（Ignition Fortress）、制御は ROS 2 Humble、環境は講座配布の Docker イメージを使用した。

方策の学習には **ACT（Action Chunking with Transformers）** による模倣学習を用い、第5回の演習（`pai_training`）をベースとした。教師データは MoveIt の公式サンプル `pick_and_place` の動作を記録・再生して自動生成し、学習した方策を Gazebo 上のシミュレーション環境にデプロイして動作を確認した。

学習済み方策により、**ロボットはブロックに到達し、掴みにいき、約18cm移動させた。**

ただし**把持は完全ではなく、運搬の途中でブロックが転がる。**教師データである MoveIt の到達位置にも届いていない。したがって本課題は、Pick and Place を**完遂したとは言えず、到達・接触・移動までの部分的な達成**である。本資料では、この到達度を前提として記述する。

動画：（YouTube URL）

---

2. 目的

本課題の目的は、AIロボット開発の一連のサイクル ― **データ収集 → 変換 → 学習 → デプロイ → 検証** ― を、途中で人に頼らず自分で一周させることに置いた。

個別の技術を深く掘るのではなく、**各工程のつなぎ目で何が起きるか**を自分の手で確かめることを優先した。結果として、本資料の中心は「うまくいった手順」ではなく「**どこで、なぜ動かなかったか**」の記録になっている。

---

3. 工夫した点

3-1. データ収集の自動化（手動テレオペの置き換え）

演習の標準手順では、キーボードによるテレオペでデモを人手収集する。これを**完全に自動化**した。

```
① record_states.py
   MoveIt の pick_and_place 実行中の /joint_states を 20Hz で CSV に記録
   （561行・29秒分）

② replay_demo.py
   その CSV を joint_trajectory トピックへ 20Hz で流し直して再生し、
   エピソードごとに rosbag を収録（15エピソードを全自動）
```

この方式を選んだ理由は2つある。

第一に、第5回講義の「データ収集のコツ：軌道を一意にする（ACT は多峰性が苦手）」への直接の回答になる。人がキーボードで操作すると軌道は必ずばらつくが、記録・再生方式では原理的にばらつきが生じない。

第二に、軌道が MoveIt の生成したものであること。収集時の Plan は15回すべて成功しており、**軌道が破綻していないこと、および15エピソードが同一の軌道であることは確認できている。** ただし、15回それぞれで把持・運搬が最後まで完了したかの確認や、変換後データの品質検査までは行っていない。この点は本課題の限界である。

当初の計画は失敗した。 最初は「MoveIt がコントローラへ送る指令を横取りして記録する」方針を立て、`/crane_plus_arm_controller/controller_state` を中継するノードを書いた。しかし収録された件数は 0 件だった。調べると、このトピックは宣言されているだけで**メッセージが一切流れていない**。MoveIt はトピックではなくアクション経由でコントローラを駆動するためである。上記の2段階方式は、この失敗を受けた代替案である。

3-2. 学習ステップ数を3水準で比較

学習ステップ数を **200 / 1000 / 10000** の3水準とし、同一のデータセット・同一の推論条件で比較した。

比較は2つの軸で行った。

- **オフライン指標**：データセット上での予測行動と正解行動の L2 誤差（`debug-policy`）
- **閉ループ性能**：実際に Gazebo 上で方策を走らせ、関節の可動域とブロックの移動距離を測定

この2軸を分けたことが、後述する本資料の主要な知見につながった。

---

4. 実装の詳細

項目	内容
ロボット	CRANE+ V2（4自由度アーム＋グリッパ）
シミュレータ	Gazebo（Ignition Fortress）
ミドルウェア	ROS 2 Humble
方策	ACT（chunk_size 16、CPU学習）
データセット	15エピソード × 559フレーム（LeRobot 形式）
タスク	5cm 木製ブロックの Pick and Place

講義資料 p.49 の指示どおり、`delta_timestamps.action` を既定の `[-0.1, 0.0, …, 1.4]` から `[0.0, 0.1, …, 1.5]` へ変更して学習した。

実装上、次の2点が落とし穴になった。

- **`ros2 bag record` には `--use-sim-time` が必須。** 付けないと bag のタイムスタンプが実時間基準になり、シミュレーションが実時間の2〜5割でしか進まないため、変換時にフレーム数が数倍に水増しされる。
- **関節の並びは `/joint_states` に合わせる必要がある。** 実データの並びは `joint1, joint2, joint4, joint3, hand` であり、joint3 と joint4 が入れ替わっている。変換スクリプトは名前ではなく**インデックス順**で `observation.state` を組むため、送信側の並びを揃えないと学習時に対応が崩れる。

---

5. 結果

5-1. 学習とオフライン評価

	200ステップ	1000ステップ	10000ステップ
学習時間（CPU）	約9分	約40分	約2時間
最終 loss	3.0	1.596	**0.069**
Mean L2 error（ep0・559フレーム）	0.7098	0.5071	**0.3542**

オフライン指標は**滑らかに改善**している。loss は約43分の1、L2 誤差は半分になった。

正解行動（破線）と予測行動（実線）を重ねて描くと、改善の様子が視覚的にも確認できる。200ステップでは予測線が正解の形すら追えていないのに対し、10000ステップでは山と谷の位置がほぼ一致する。

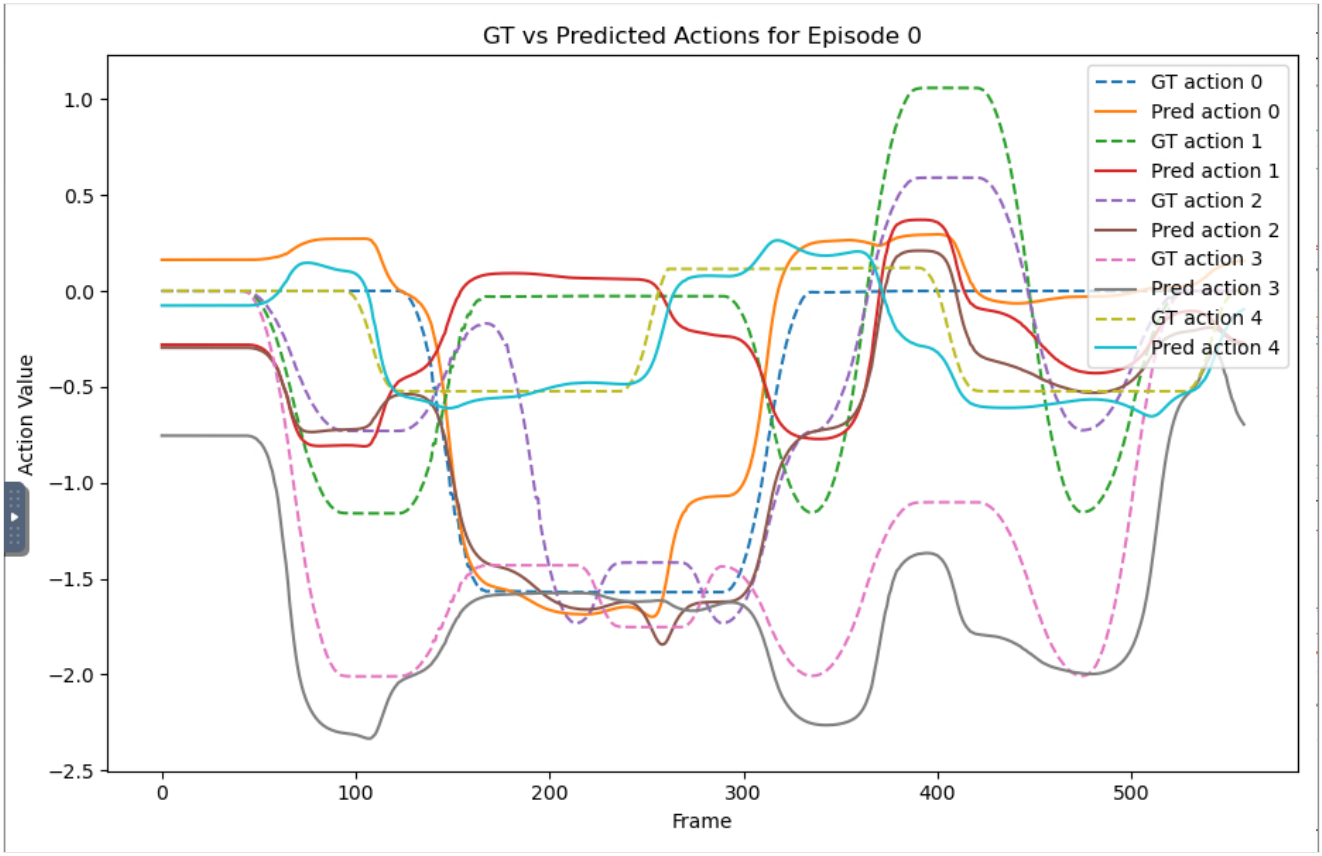


図1　200ステップ

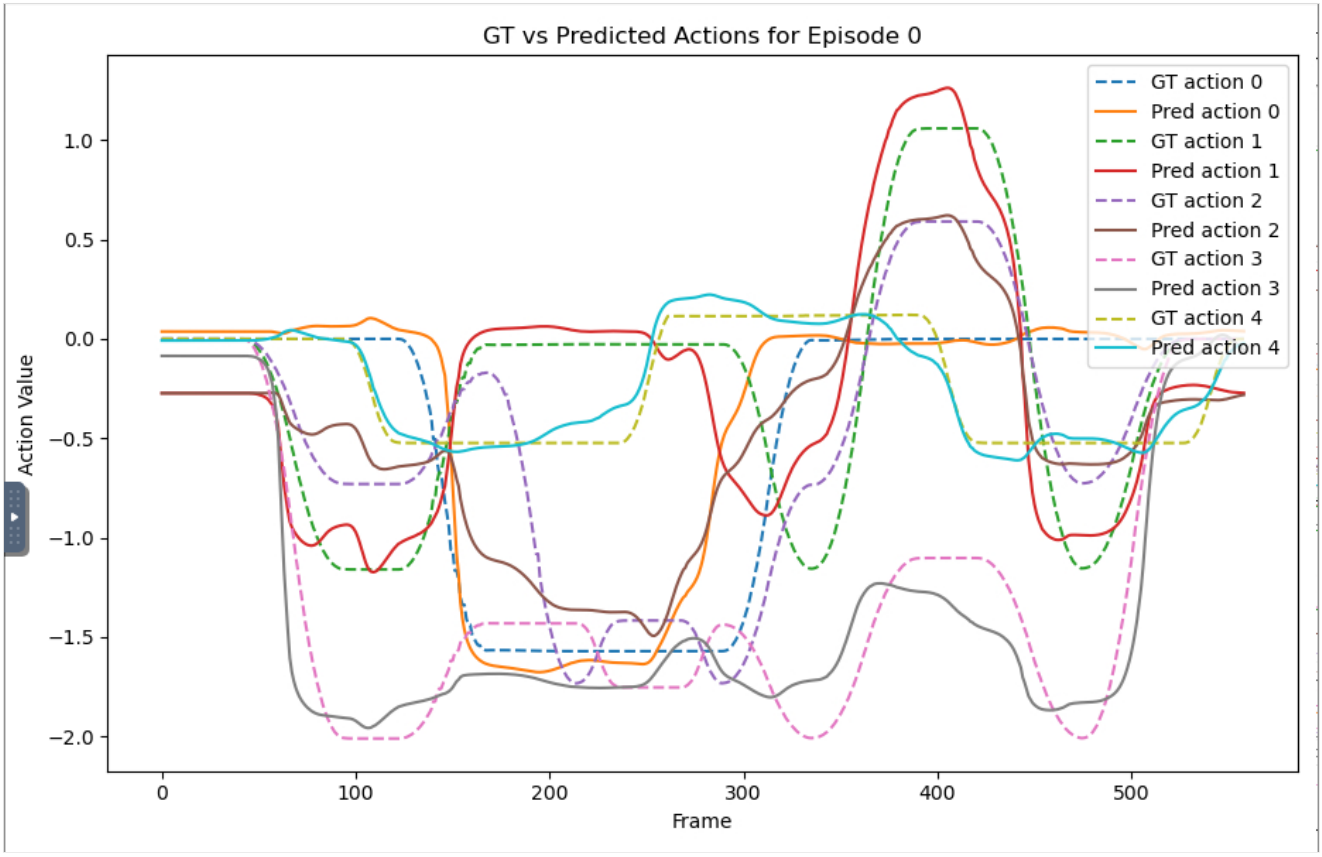


図2　1000ステップ

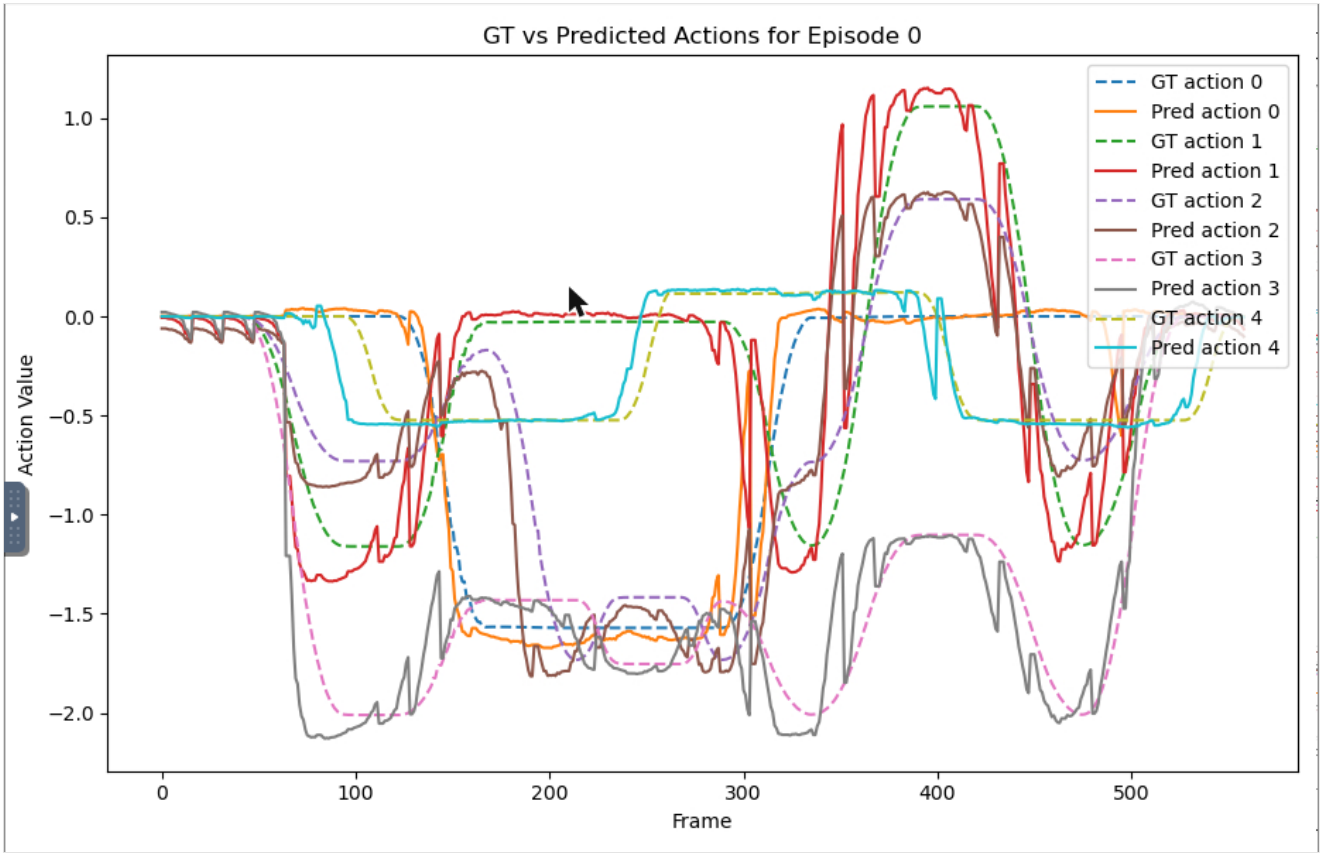


図3　10000ステップ（破線＝正解、実線＝予測。横軸はフレーム、縦軸は関節の目標角）

5-2. 閉ループ性能

同一条件（初期姿勢をゼロに揃え、ブロックを定位置に戻し、55秒間実行）で測定した。

到達度の判定基準は、本資料では次の3段階とする。要項に完成度の合否基準が定められていないため、こちらで定義した。

段階	定義
① 未到達	ブロックが動かない（0cm）
② 部分達成	ブロックに接触し、移動させる
③ 完遂	把持を保ったまま教師データの目標位置まで運ぶ

	200	1000	10000
joint1（基部旋回）の可動域	0.272 〜 0.275	-0.113 〜 0.103	**-1.662 〜 0.036**
グリッパ	0.161 〜 0.166	-0.604 〜 -0.085	**-0.557 〜 +0.134**
アームの挙動	完全に静止	動くが旋回しない	**軌道を完走**
ブロックの移動距離	**0 cm**	**0 cm**	**18.2 cm**

ブロックの最終位置：

```
200   : (0.000, -0.100)   動かず
1000  : (0.000, -0.100)   動かず
10000 : (0.136,  0.021)   18.2cm 移動
```

教師データである MoveIt の原典は (0.286, 0.015) まで運ぶため、10000ステップ版もそこには届いていない。把持が完全ではなく、運搬の途中でブロックが転がる。

上記の基準に照らすと、**200 と 1000 は①未到達、10000 は②部分達成**である。③完遂には至っていない。

---

6. 考察

6-1. オフライン指標の改善は、ロボットが物体を動かすことを保証しなかった

本課題で最も重要な発見である。

L2 誤差は 0.71 → 0.51 → 0.35 と**連続的に**改善する。しかし物体が動いたかどうかは **0cm / 0cm / 18.2cm** と**不連続**だった。200 から 1000 で誤差が約3割改善しても、ロボットは1ミリも仕事をしていない。

数値が改善していることと、ロボットが仕事をすることは、別の事象である。オフライン評価だけを見て「学習は進んでいる」と判断すると、デプロイして初めて何も動かないことに気づくことになる。実際、本課題では一度そうなった。

この主張の範囲について補足する。 本課題で確認できたのは「3水準の比較において、オフライン指標の改善だけでは物体の移動を保証しなかった」という事実であり、両者の関係一般を確定したものではない。水準が3点であること、③完遂には至っていないこと、評価がシミュレーション上の1タスクに限られることから、この結果をもって一般則を主張することはできない。

6-2. 閉ループでしか現れない欠陥が3つあった

10000ステップまで学習させても、最初は**ロボットが動かなかった**。原因は学習側ではなく、すべて推論側にあった。

	欠陥	症状	対処
1	初期姿勢が学習データと異なる	分布外の状態から始まり方策が破綻	推論前に全関節をゼロ姿勢へ送る
2	`n_action_steps: 1`	固定点に落ちて完全停止	16（= `chunk_size`）に変更
3	20Hz で publish	軌道が倍速になり追従できない	10Hz（timer 0.1秒）に変更

欠陥2 は推論の構造に起因する。`n_action_steps: 1` は毎制御ステップごとに現在の関節角から推論をやり直す設定である。ACT は過去を持たないため、

```
状態 S → 行動 A を出力 → ロボットが A へ移動 → 状態が A になる → また A を出力
```

という自己一致点にはまり、動かなくなる。

欠陥3 は時間スケールの不一致である。`delta_timestamps.action` が `[0.0, 0.1, …, 1.5]` であるとおり、行動チャンクの16点は **0.1秒間隔**で学習されている。ところが `infer.py` は 0.05秒間隔（20Hz）で publish していた。軌道を**倍速で再生していた**ことになる。アームは追従できず実際の姿勢が遅れ続け、方策は「まだ軌道の前半にいる」状態を観測して前半のチャンクを出し直す。これが無限ループの正体だった。修正後、joint1 の可動域は -0.223 から **-1.666** へ一変し、ブロックが動くようになった。

3つとも、オフラインの `debug-policy` では一切検出できない。 観測がデータセット側から順に与えられるため、閉ループ特有の問題が表面化しないからである。

MLOps / RobOps の文脈で「検証のサイクル」と言うとき、そこにはオフライン評価と**方策を実際に動かした評価**の両方が含まれていなければならない、というのが本課題で得た実感である。なお本課題の評価はすべて Gazebo 上のシミュレーションであり、実機では検証していない。

6-3. 学習量と推論の正しさは、どちらか一方では足りない

推論側の欠陥をすべて修正した上でも、200 と 1000 はブロックに触れもしなかった。逆に、10000ステップまで学習しても推論側を直さなければ動かなかった。

両方が揃って初めて物体が動いた。 もし推論の欠陥を抱えたまま 200 と 10000 を比較していれば、どちらも動かないため「学習量は関係ない」という正反対の結論に至っていた。条件を揃えた比較の重要性を、身をもって確認した。

---

7. 失敗の記録

本課題では、一度**誤った結論に到達して方針転換しかけた。**

グリッパがブロックを弾いてしまう現象が続いたため、「CRANE+ V2 のグリッパは開口が5cmの立方体に足りず、このシミュレーション環境では把持は原理的に不可能」と結論し、タスクを「押す」に変更した。さらに、12cm の背の高い箱を環境に追加することで物体操作を成立させた。

しかし、CRANE+ V2 はメーカーの公式デモで実機が普通に pick and place を行っている。「できない」のはこちら側の問題ではないかと考え直し、**一切手を加えていない素のシミュレーションで公式サンプルを実行した。結果は、Plan 15回すべて成功**だった。

失敗の原因は自分の使い方にあった。

1. 実行前にテレオペでアームを不自然な姿勢にしていた
2. MoveIt を使わず `JointTrajectory` を単一点で publish していた（MoveIt は経由点を密に打つため追従が良い。単一点では重力でアームが垂れる）
3. `use_sim_time:=true` を付けていなかった

環境やハードウェアの限界だと判断する前に、**標準的な手順をそのまま実行して確かめる**こと。これを怠ったために、不要な方針転換と環境改変を行った。背の高い箱も、トルク上限の変更も、すべて不要だった。

---

8. 環境とコード

ベースイメージ	`airobotbook/ros2-desktop-ai-robot-book-humble`
ベースリポジトリ	`matsuolab/pai_training`（第5回演習）
パッケージ管理	pixi
実行環境	Apple Silicon Mac（Docker・CPU学習、GPU不使用）

コード一式：（GitHub URL）

主なファイル（**自作**／**改変**／**配布物そのまま**を明示する）：

ファイル	由来	内容
`record_states.py`	**自作**	MoveIt のデモ軌道を `/joint_states` から CSV に記録
`replay_demo.py`	**自作**	CSV を再生し、エピソードごとに rosbag を収録（全自動）
`mon.py`	**自作**	関節の可動域を一定時間測定（閉ループ検証用）
`scripts/infer.py`	**配布物を改変**	推論ノード。publish 周期を 20Hz → 10Hz に変更
`configs/train_config.yaml`	**配布物を改変**	`delta_timestamps`・学習ステップ数・出力先
`configs/conversion_config.yaml`	**配布物を改変**	bag とデータセットのパス
`configs/crane_features.py`	**配布物を改変**	`repo_id` / `task_name`
`scripts/train.py` 他	配布物（未改変）	学習・変換・評価
`pyproject.toml`	配布物（未改変）	依存関係と pixi タスク定義

改変したファイルは、**改変前の原本も `.bak` / `.orig` として併せて収録**している（`scripts/infer.py.bak`、`configs/train_config.yaml.orig`）。差分がそのまま確認できる。

学習済みモデルのチェックポイントについては、推論時に `config.json` の `n_action_steps` を 1 → 16 に変更している（6-2 の欠陥2への対処）。こちらも `.bak` を残している。

---

9. まとめ

- MoveIt のデモ動作を記録・再生することで、**手作業ゼロで15エピソードの教師データを収集**した
- 学習ステップ数 200 / 1000 / 10000 を比較し、**10000ステップで②部分達成**に至った（ブロックに到達・接触し、18.2cm移動）。③完遂（把持を保って目標位置まで運ぶ）には届いていない
- **オフライン指標の改善だけでは、ロボットが物体を動かすことを保証しなかった。** L2 誤差は 0.71 → 0.51 → 0.35 と連続的に改善するのに対し、物体の移動は 0cm / 0cm / 18.2cm と不連続だった。ただし3水準の比較であり、一般則として主張するものではない

- 閉ループでしか現れない欠陥を3つ特定し、修正した。いずれもオフライン評価では検出不能だった
- 環境の限界だと判断する前に、標準手順をそのまま試す。これを怠って一度誤った結論に至った
- 本課題の評価はすべて Gazebo 上のシミュレーションであり、実機での検証は行っていない